6.5840 - Spring 2026

6.5840 Lab 3: Raft

Introduction

This is the first in a series of labs in which you'll build a fault-tolerant key/value storage system. In this lab you'll implement Raft, a replicated state machine protocol. In the next lab you'll build a key/value service on top of Raft. Then you will “shard” your service over multiple replicated state machines for higher performance.

A replicated service achieves fault tolerance by storing complete copies of its state (i.e., data) on multiple replica servers. Replication allows the service to continue operating even if some of its servers experience failures (crashes or a broken or flaky network). The challenge is that failures may cause the replicas to hold differing copies of the data.

Raft organizes client requests into a sequence, called the log, and ensures that all the replica servers see the same log. Each replica executes client requests in log order, applying them to its local copy of the service's state. Since all the live replicas see the same log contents, they all execute the same requests in the same order, and thus continue to have identical service state. If a server fails but later recovers, Raft takes care of bringing its log up to date. Raft will continue to operate as long as at least a majority of the servers are alive and can talk to each other. If there is no such majority, Raft will make no progress, but will pick up where it left off as soon as a majority can communicate again.

In this lab you'll implement Raft as a Go object type with associated methods, meant to be used as a module in a larger service. A set of Raft instances talk to each other with RPC to maintain replicated logs. Your Raft interface will support an indefinite sequence of numbered commands, also called log entries. The entries are numbered with *index numbers*. The log entry with a given index will eventually be committed. At that point, your Raft should send the log entry to the larger service for it to execute.

You should follow the design in the extended Raft paper, with particular attention to Figure 2. You'll implement most of what's in the paper, including saving persistent state and reading it after a node fails and then restarts. You will not implement cluster membership changes (Section 6).

This lab is due in four parts. You must submit each part on the corresponding due date.

Getting Started

If you have done Lab 1, you already have a copy of the lab source code. If not, you can find directions for obtaining the source via git in the Lab 1 instructions.

We supply you with skeleton code `src/raft1/raft.go`. We also supply a set of tests, which you should use to drive your implementation efforts, and which we'll use to grade your submitted lab. The tests are in `src/raft1/raft_test.go`.

When we grade your submissions, we will run the tests without the `-race` flag. However, you should test with `-race`.

To get up and running, execute the following commands. Don't forget the `git pull` to get the latest software.

```
$ cd ~/6.5840
$ git pull
...
$ cd src
$ make raft1
go build -race -o main/raft1d main/raft1d.go
cd raft1 && go test -v -race
=== RUN   TestInitialElection3A
Test (3A): initial election (reliable network)...
Fatal: expected one leader, got none
        /Users/rtm/824-process-raft/src/raft1/test.go:151
        /Users/rtm/824-process-raft/src/raft1/raft_test.go:36
info: wrote visualization to /var/folders/x_/vk0xmxwn1sj91m89wsn5b1yh0000gr/T/porcupi
--- FAIL: TestInitialElection3A (5.51s)
...
$
```

The code

Implement Raft by adding code to `raft1/raft.go`. In that file you'll find skeleton code, plus examples of how to send and receive RPCs.

Your implementation must support the following interface, which the tester and (eventually) your key/value server will use. You'll find more details in comments in `raft.go` and in `raftapi/raftapi.go`.

```
// create a new Raft server instance:
rf := Make(peers, me, persister, applyCh)

// start agreement on a new log entry:
rf.Start(command interface{}) (index, term, isleader)

// ask a Raft for its current term, and whether it thinks it is leader
rf.GetState() (term, isLeader)

// each time a new entry is committed to the log, each Raft peer
// should send an ApplyMsg to the service (or tester).
type ApplyMsg
```

A service calls `Make(peers,me,…)` to create a Raft peer. The peers argument is an array of network identifiers of the Raft peers (including this one), for use with RPC. The `me` argument is the index of this peer in the peers array. `Start(command)` asks Raft to start the processing to append the command to the replicated log. `Start()` should return immediately, without waiting for the log appends to complete. The service expects your implementation to send an `ApplyMsg` for each newly committed log entry to the `applyCh` channel argument to `Make()`.

`raft.go` contains example code that sends an RPC (`sendRequestVote()`) and that handles an incoming RPC (`RequestVote()`). Your Raft peers should exchange RPCs using the labrpc Go package (source in `src/labrpc`). The tester can tell `labrpc` to delay RPCs, re-order them, and discard them to simulate various network failures. While you can temporarily modify `labrpc`, make sure your Raft works with the original `labrpc`, since that's what we'll use to test and grade your lab. Your Raft instances must interact only through RPC; for example, they are not allowed to communicate using shared Go variables or files.

Subsequent labs build on this lab, so it is important to give yourself enough time to write solid code.

Part 3A: leader election (moderate)

TASK

Implement Raft leader election and heartbeats (`AppendEntries` RPCs with no log entries). The goal for Part 3A is for a single leader to be elected, for the leader to remain the leader if there are no failures, and for a new leader to take over if the old leader fails or if packets to/from the old leader are lost. Run `make RUN="-run 3A" raft1` in the `src` directory to test your 3A code.

- **Hint:** Follow the paper's Figure 2. At this point you care about sending and receiving RequestVote RPCs, the Rules for Servers that relate to elections, and the State related to leader election,
- **Hint:** Add the Figure 2 state for leader election to the `Raft` struct in `raft.go`.
- **Hint:** Fill in the `RequestVoteArgs` and `RequestVoteReply` structs. Modify `Make()` to create a background goroutine that will kick off leader election periodically by sending out `RequestVote` RPCs when it hasn't heard from another peer for a while. Implement the `RequestVote()` RPC handler so that servers will vote for one another.
- **Hint:** To implement heartbeats, define an `AppendEntries` RPC struct (though you may not need all the arguments yet), and have the leader send them out periodically. Write an `AppendEntries` RPC handler method.
- **Hint:** The tester requires that the leader send heartbeat RPCs no more than ten times per second.
- **Hint:** The tester requires your Raft to elect a new leader within five seconds of the failure of the old leader (if a majority of peers can still communicate).
- **Hint:** The paper's Section 5.2 mentions election timeouts in the range of 150 to 300 milliseconds. Such a range only makes sense if the leader sends heartbeats considerably more often than once per 150 milliseconds (e.g., once per 10 milliseconds). Because the tester limits you tens of heartbeats per second, you will have to use an election timeout larger than the paper's 150 to 300 milliseconds, but not too large, because then you may fail to elect a leader within five seconds.
- **Hint:** You may find Go's rand useful.
- **Hint:** You'll need to write code that takes actions periodically or after delays in time. The easiest way to do this is to create a goroutine with a loop that calls time.Sleep(); see the `ticker()` goroutine that `Make()` creates for this purpose. Don't use Go's `time.Timer` or `time.Ticker`, which are difficult to use correctly.
- **Hint:** If your code has trouble passing the tests, read the paper's Figure 2 again; the full logic for leader election is spread over multiple parts of the figure.
- **Hint:** Don't forget to implement `GetState()`.
- **Hint:** Go RPC sends only struct fields whose names start with capital letters. Sub-structures must also have capitalized field names (e.g. fields of log records in an array). The `labgob` package will warn you about this; don't ignore the warnings.
- **Hint:** The most challenging part of this lab may be the debugging. Refer to the Guidance page for debugging tips.
- **Hint:** If you fail a test, the tester produces a file that visualizes a timeline with events marked along it, including network partitions, crashed servers, and checks performed. Here's an example of the visualization. Further, you can add your own annotations by writing, for example, `tester.Annotate("Server 0", "short description", "details")`.

Be sure you pass the 3A tests before submitting Part 3A, so that you see something like this:

```
$ make RUN="-run 3A" raft1
go build -race -o main/raft1d main/raft1d.go
cd raft1 && go test -v -race -run 3A
=== RUN   TestInitialElection3A
Test (3A): initial election (reliable network)...
  ... Passed --  time  3.5s #peers 3 #RPCs    32 #Ops    0
--- PASS: TestInitialElection3A (3.84s)
```

```
=== RUN   TestReElection3A
Test (3A): election after network failure (reliable network)...
  ... Passed --  time  6.2s #peers 3 #RPCs    68 #Ops    0
--- PASS: TestReElection3A (6.54s)
=== RUN   TestManyElections3A
Test (3A): multiple elections (reliable network)...
  ... Passed --  time  9.8s #peers 7 #RPCs   684 #Ops    0
--- PASS: TestManyElections3A (10.68s)
PASS
ok      6.5840/raft1    22.095s
$
```

Each "Passed" line contains five numbers; these are the time that the test took in seconds, the number of Raft peers, the number of RPCs sent during the test, the total number of bytes in the RPC messages, and the number of log entries that Raft reports were committed. Your numbers will differ from those shown here. You can ignore the numbers if you like, but they may help you sanity-check the number of RPCs that your implementation sends. For all of labs 3, 4, and 5, the grading script will fail your solution if it takes more than 600 seconds for all of the tests, or if any individual test takes more than 120 seconds.

When we grade your submissions, we will run the tests without the `-race` flag. However, you should make sure that your code consistently passes the tests with the `-race` flag.

Part 3B: log (hard)

TASK

Implement the leader and follower code to append new log entries, so that `make RUN="-run 3B" raft1` passes all tests.

- **Hint:** Run `git pull` to get the latest lab software.
- **Hint:** The Raft paper views the log as 1-indexed, but we suggest that you implement it as 0-indexed, starting with a dummy entry at index=0 that has term 0. That allows the very first AppendEntries RPC to contain 0 as PrevLogIndex, and be a valid index into the log.
- **Hint:** Your first goal should be to pass `TestBasicAgree3B()`. Start by implementing `Start()`, then write the code to send and receive new log entries via `AppendEntries` RPCs, following Figure 2. Send each newly committed entry on `applyCh` on each peer.
- **Hint:** You will need to implement the election restriction (section 5.4.1 in the paper).
- **Hint:** Your code may have loops that repeatedly check for certain events. Don't have these loops execute continuously without pausing, since that will slow your implementation enough that it fails tests. Use Go's condition variables, or insert a `time.Sleep(10 * time.Millisecond)` in each loop iteration.
- **Hint:** Do yourself a favor for future labs and write (or re-write) code that's clean and clear.
- **Hint:** If you fail a test, look at `raft_test.go` and trace the test code from there to understand what's being tested.

The tests for upcoming labs may fail your code if it runs too slowly. You can check how much real time and CPU time your solution uses with the time command. Here's typical output:

```
$ make RUN="-run 3B" raft1
go build -race -o main/raft1d main/raft1d.go
cd raft1 && go test -v -race -run 3B
=== RUN   TestBasicAgree3B
Test (3B): basic agreement (reliable network)...
  ... Passed --  time  1.6s #peers 3 #RPCs    18 #Ops    3
--- PASS: TestBasicAgree3B (1.96s)
```

```
=== RUN   TestRPCBytes3B
Test (3B): RPC byte count (reliable network)...
  ... Passed --  time  3.3s #peers 3 #RPCs    50 #Ops   11
--- PASS: TestRPCBytes3B (3.71s)
=== RUN   TestFollowerFailure3B
Test (3B): test progressive failure of followers (reliable network)...
  ... Passed --  time  5.4s #peers 3 #RPCs    58 #Ops    3
--- PASS: TestFollowerFailure3B (5.77s)
=== RUN   TestLeaderFailure3B
Test (3B): test failure of leaders (reliable network)...
  ... Passed --  time  6.5s #peers 3 #RPCs   110 #Ops    3
--- PASS: TestLeaderFailure3B (6.89s)
=== RUN   TestFailAgree3B
Test (3B): agreement after follower reconnects (reliable network)...
  ... Passed --  time  6.0s #peers 3 #RPCs    61 #Ops    7
--- PASS: TestFailAgree3B (6.37s)
=== RUN   TestFailNoAgree3B
Test (3B): no agreement if too many followers disconnect (reliable network)...
  ... Passed --  time  4.0s #peers 5 #RPCs   107 #Ops    2
--- PASS: TestFailNoAgree3B (4.55s)
=== RUN   TestConcurrentStarts3B
Test (3B): concurrent Start()s (reliable network)...
  ... Passed --  time  1.4s #peers 3 #RPCs    12 #Ops    0
--- PASS: TestConcurrentStarts3B (1.75s)
=== RUN   TestRejoin3B
Test (3B): rejoin of partitioned leader (reliable network)...
  ... Passed --  time  7.8s #peers 3 #RPCs   120 #Ops    4
--- PASS: TestRejoin3B (8.15s)
=== RUN   TestBackup3B
Test (3B): leader backs up quickly over incorrect follower logs (reliable network)...
  ... Passed --  time 27.7s #peers 5 #RPCs  1370 #Ops  102
--- PASS: TestBackup3B (28.27s)
=== RUN   TestCount3B
Test (3B): RPC counts aren't too high (reliable network)...
  ... Passed --  time  2.7s #peers 3 #RPCs    32 #Ops    0
--- PASS: TestCount3B (3.05s)
PASS
ok      6.5840/raft1    71.716s
$
```

The "ok 6.5840/raft 71.716s" means that Go measured the time taken for the 3B tests to be 71.716 seconds of real (wall-clock) time. If your solution uses much more than a few minutes of real time for the 3B tests, you may run into trouble later on. Look for time spent sleeping or waiting for RPC timeouts, loops that run without sleeping or waiting for conditions or channel messages, or large numbers of RPCs sent.

Part 3C: persistence (hard)

If a Raft-based server reboots it should resume service where it left off. This requires that Raft keep persistent state that survives a reboot. The paper's Figure 2 mentions which state should be persistent.

A real implementation would write Raft's persistent state to disk each time it changed, and would read the state from disk when restarting after a reboot. Your implementation won't use the disk; instead, it will save and restore persistent state from a `Persister` object (see `tester1/persister.go`). Whoever calls `Raft.Make()` supplies a `Persister` that initially holds Raft's most recently persisted state (if any). Raft should initialize its state from that `Persister`, and should use it to save its persistent state each time the state changes. Use the `Persister`'s `ReadRaftState()` and `Save()` methods.

TASK

Complete the functions `persist()` and `readPersist()` in `raft.go` by adding code to save and restore persistent state. You will need to encode (or "serialize") the state as an array of bytes in order to pass it to the `Persister`. Use the

`labgob` encoder; see the comments in `persist()` and `readPersist()`. `labgob` is like Go's `gob` encoder but prints error messages if you try to encode structures with lower-case field names. For now, pass `nil` as the second argument to `persister.Save()`. Insert calls to `persist()` at the points where your implementation changes persistent state. Once you've done this, and if the rest of your implementation is correct, you should pass all of the 3C tests.

You will probably need the optimization that backs up nextIndex by more than one entry at a time. Look at the extended Raft paper starting at the bottom of page 7 and top of page 8 (marked by a gray line). The paper is vague about the details; you will need to fill in the gaps. One possibility is to have a rejection message include:

```
    XTerm:  term in the conflicting entry (if any)
    XIndex: index of first entry with that term (if any)
    XLen:   log length
```

Then the leader's logic can be something like:

```
  Case 1: leader doesn't have XTerm:
    nextIndex = XIndex
  Case 2: leader has XTerm:
    nextIndex = (index of leader's last entry for XTerm) + 1
  Case 3: follower's log is too short:
    nextIndex = XLen
```

A few other hints:

- **Hint:** Run `git pull` to get the latest lab software.
- **Hint:** The 3C tests are more demanding than those for 3A or 3B, and failures may be caused by problems in your code for 3A or 3B.

Your code should pass all the 3C tests (as shown below), as well as the 3A and 3B tests.

```
$ make RUN="-run 3C" raft1
go build -race -o main/raft1d main/raft1d.go
cd raft1 && go test -v -race -run 3C
=== RUN   TestPersist13C
Test (3C): basic persistence (reliable network)...
  ... Passed --  time  7.6s #peers 3 #RPCs    58 #Ops    6
--- PASS: TestPersist13C (7.99s)
=== RUN   TestPersist23C
Test (3C): more persistence (reliable network)...
  ... Passed --  time 21.6s #peers 5 #RPCs   287 #Ops   16
--- PASS: TestPersist23C (22.17s)
=== RUN   TestPersist33C
Test (3C): partitioned leader and one follower crash, leader restarts (reliable netwo
  ... Passed --  time  3.8s #peers 3 #RPCs    30 #Ops    4
--- PASS: TestPersist33C (4.11s)
=== RUN   TestFigure83C
Test (3C): Figure 8 (reliable network)...
  ... Passed --  time 48.5s #peers 5 #RPCs   499 #Ops    2
--- PASS: TestFigure83C (49.08s)
=== RUN   TestUnreliableAgree3C
Test (3C): unreliable agreement (unreliable network)...
  ... Passed --  time  5.1s #peers 5 #RPCs   288 #Ops  246
--- PASS: TestUnreliableAgree3C (5.68s)
=== RUN   TestFigure8Unreliable3C
Test (3C): Figure 8 (unreliable) (unreliable network)...
```

```
  ... Passed --  time 53.6s #peers 5 #RPCs  3200 #Ops    2
--- PASS: TestFigure8Unreliable3C (54.19s)
=== RUN   TestReliableChurn3C
Test (3C): churn (reliable network)...
  ... Passed --  time 18.2s #peers 5 #RPCs  1701 #Ops    1
--- PASS: TestReliableChurn3C (18.80s)
=== RUN   TestUnreliableChurn3C
Test (3C): unreliable churn (unreliable network)...
  ... Passed --  time 17.3s #peers 5 #RPCs  1253 #Ops    1
--- PASS: TestUnreliableChurn3C (17.92s)
PASS
ok      6.5840/raft1    180.983s
$
```

It is a good idea to run the tests multiple times before submitting.

Part 3D: log compaction (hard)

As things stand now, a rebooting server replays the complete Raft log in order to restore its state. However, it's not practical for a long-running service to remember the complete Raft log forever. Instead, you'll modify Raft to cooperate with services that persistently store a "snapshot" of their state from time to time, at which point Raft discards log entries that precede the snapshot. The result is a smaller amount of persistent data and faster restart. However, it's now possible for a follower to fall so far behind that the leader has discarded the log entries it needs to catch up; the leader must then send a snapshot plus the log starting at the time of the snapshot. Section 7 of the extended Raft paper outlines the scheme; you will have to design the details.

Your Raft must provide the following function that the service can call with a serialized snapshot of its state:

```
Snapshot(index int, snapshot []byte)
```

In Lab 3D, the tester calls `Snapshot()` periodically. In Lab 4, you will write a key/value server that calls `Snapshot()`; the snapshot will contain the complete table of key/value pairs. The service layer calls `Snapshot()` on every peer (not just on the leader).

The `index` argument indicates the highest log entry that's reflected in the snapshot. Raft should discard its log entries before that point. You'll need to revise your Raft code to operate while storing only the tail of the log.

You'll need to implement the `InstallSnapshot` RPC discussed in the paper that allows a Raft leader to tell a lagging Raft peer to replace its state with a snapshot. You will likely need to think through how InstallSnapshot should interact with the state and rules in Figure 2.

When a follower's Raft code receives an InstallSnapshot RPC, it can use the `applyCh` to send the snapshot to the service in an `ApplyMsg`. The `ApplyMsg` struct definition in `raftapi/raftapi.go` already contains the fields you will need (and which the tester expects). Take care that these snapshots only advance the service's state, and don't cause it to move backwards.

If a server crashes, it must restart from persisted data. Your Raft should persist both Raft state and the corresponding snapshot. Use the second argument to `persister.Save()` to save the snapshot. If there's no snapshot, pass `nil` as the second argument.

When a server restarts, the application layer reads the persisted snapshot and restores its saved application state. After a restart, the application layer expects the first message on applyCh to either contain a snapshot with a SnapshotIndex higher than that of the initial restored snapshot, or an ordinary command with CommandIndex immediately following the index of the initial restored snapshot.

TASK

Implement `Snapshot()` and the InstallSnapshot RPC, as well as the changes to Raft to support these (e.g, operation with a trimmed log). Your solution is complete when it passes the 3D tests (and all the previous Lab 3 tests).

- **Hint:** `git pull` to make sure you have the latest software.
- **Hint:** A good place to start is to modify your code to so that it is able to store just the part of the log starting at some index X. Initially you can set X to zero and run the 3B/3C tests. Then make `Snapshot(index)` discard the log before `index`, and set X equal to `index`. If all goes well you should now pass the first 3D test.
- **Hint:** A common reason for failing the first 3D test is that followers take too long to catch up to the leader.
- **Hint:** Next: have the leader send an InstallSnapshot RPC if it doesn't have the log entries required to bring a follower up to date.
- **Hint:** Send the entire snapshot in a single InstallSnapshot RPC. Don't implement Figure 13's `offset` mechanism for splitting up the snapshot.
- **Hint:** Raft must discard old log entries in a way that allows the Go garbage collector to free and re-use the memory; this requires that there be no reachable references (pointers) to the discarded log entries.
- **Hint:** When a Raft peer is re-started, the persister passed to `Make()` will contain a snapshot of application state as well as Raft's saved state. Raft must include a non-nil snapshot with every call to `persister.Save()` (if the log has been trimmed), which means that it's a good idea for `Make()` to call `persister.ReadSnapshot()` and save the result.
- **Hint:** A reasonable amount of time to consume for the full set of Lab 3 tests (3A+3B+3C+3D) without `-race` is 6 minutes of real time and one minute of CPU time. When running with `-race`, it is about 10 minutes of real time and two minutes of CPU time.

Your code should pass all the 3D tests (as shown below), as well as the 3A, 3B, and 3C tests.

```
$ make RUN="-run 3D" raft1
go build -race -o main/raft1d main/raft1d.go
cd raft1 && go test -v -race -run 3D
=== RUN   TestSnapshotBasic3D
Test (3D): snapshots basic (reliable network)...
  ... Passed --  time  8.4s #peers 3 #RPCs   279 #Ops   31
--- PASS: TestSnapshotBasic3D (8.74s)
=== RUN   TestSnapshotInstall3D
Test (3D): install snapshots (disconnect) (reliable network)...
  ... Passed --  time 59.6s #peers 3 #RPCs   919 #Ops   91
--- PASS: TestSnapshotInstall3D (59.99s)
=== RUN   TestSnapshotInstallUnreliable3D
Test (3D): install snapshots (disconnect) (unreliable network)...
  ... Passed --  time 82.1s #peers 3 #RPCs  1083 #Ops   91
--- PASS: TestSnapshotInstallUnreliable3D (82.49s)
=== RUN   TestSnapshotInstallCrash3D
Test (3D): install snapshots (crash) (reliable network)...
  ... Passed --  time 53.6s #peers 3 #RPCs   685 #Ops   91
--- PASS: TestSnapshotInstallCrash3D (53.99s)
=== RUN   TestSnapshotInstallUnCrash3D
Test (3D): install snapshots (crash) (unreliable network)...
  ... Passed --  time 66.2s #peers 3 #RPCs   717 #Ops   91
--- PASS: TestSnapshotInstallUnCrash3D (66.60s)
=== RUN   TestSnapshotAllCrash3D
Test (3D): crash and restart all servers (unreliable network)...
  ... Passed --  time 20.4s #peers 3 #RPCs   244 #Ops   45
--- PASS: TestSnapshotAllCrash3D (20.79s)
=== RUN   TestSnapshotInit3D
```

```
Test (3D): snapshot initialization after crash (unreliable network)...
  ... Passed --  time  7.4s #peers 3 #RPCs    79 #Ops   14
--- PASS: TestSnapshotInit3D (7.77s)
PASS
ok      6.5840/raft1    301.406s
$
```